

Messaging Feature

▼ Relevant source files

Frontend/STUDER/src/app/features/messages/components/messages/messages.component.css

Frontend/STUDER/src/app/features/messages/components/messages/messages.component.html

Frontend/STUDER/src/app/features/messages/components/messages/messages.component.ts

Frontend/STUDER/src/app/features/messages/messages.model.ts

Frontend/STUDER/src/app/features/messages/messages.service.ts

The Messaging Feature provides a private communication channel between users. It implements a friend-gated model where full conversation capabilities are reserved for confirmed friends, while initial interactions from non-friends are handled through a message request flow.

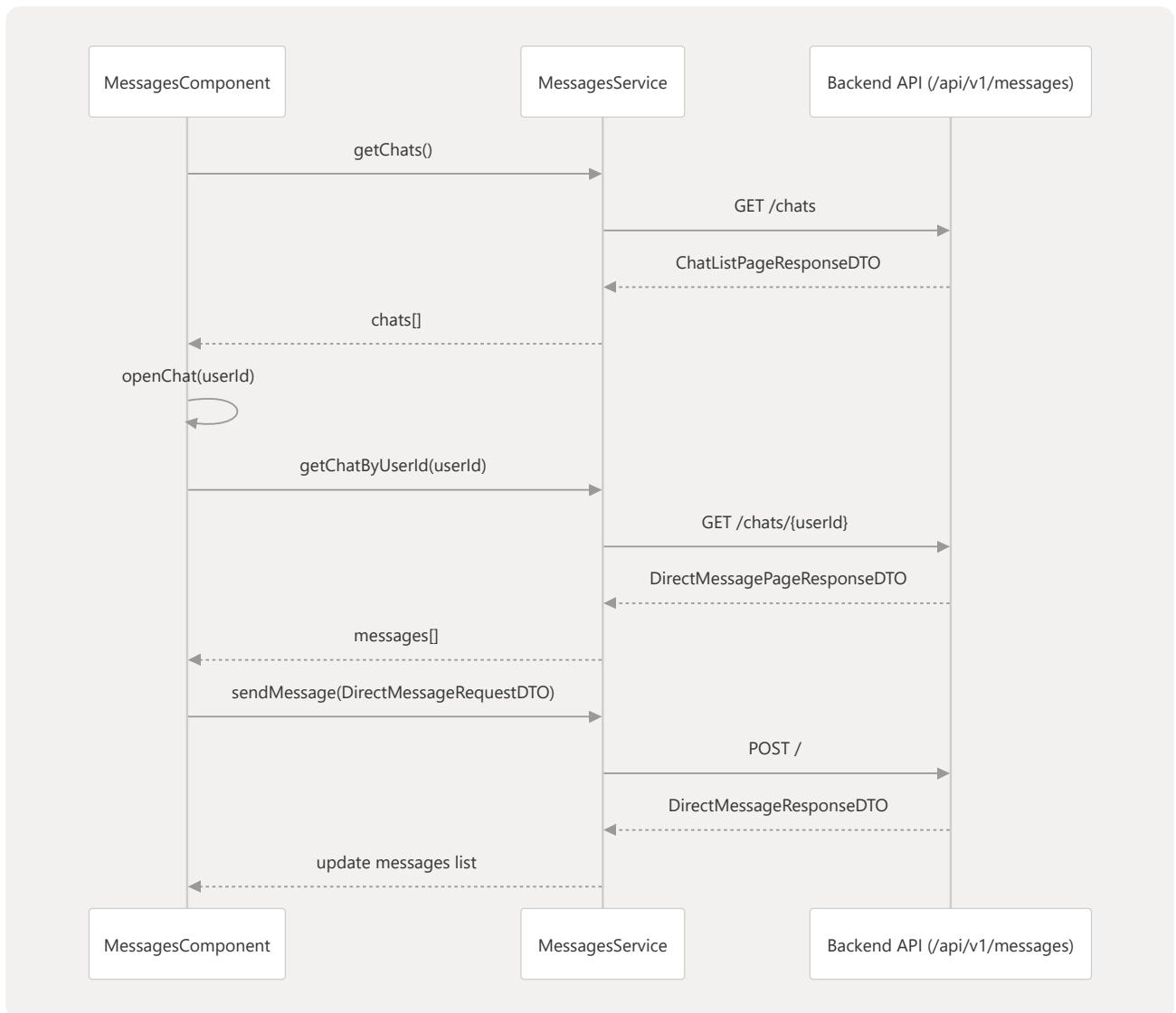
System Architecture

The messaging system is built around the `MessagesService` which interacts with the backend REST API, and the `MessagesComponent` which manages the split-pane UI for conversation lists and message threads.

Messaging Data Flow

The following diagram illustrates how the frontend components and services interact with the backend to fetch and send messages.

Messaging Interaction Diagram



Sources: Frontend/STUDER/src/app/features/messages/messages.service.ts 18-36

Frontend/STUDER/src/app/features/messages/components/messages/messages.component.ts 119-169

Data Models

The messaging feature uses specific DTOs (Data Transfer Objects) to manage conversation previews and individual message responses.

ChatPreviewDTO

Used in the sidebar to display a summary of an ongoing conversation.

- **Path:** Frontend/STUDER/src/app/features/messages/messages.model.ts 1-12
- **Key Fields:** `userId`, `username`, `lastMessageContent`, `unreadCount`, and `profilePictureThumbnailUrl`.

DirectMessageResponseDTO

Represents an individual message within a thread.

- **Path:** `Frontend/STUDER/src/app/features/messages/messages.model.ts` 21-33
- **Key Fields:** `senderId`, `content`, `sentAt`, `isRead`, and `replyToId` for threaded replies.

Component Logic

The `MessagesComponent` acts as the primary controller for the messaging UI. It handles tab switching between active chats and message requests, as well as searching through friends and existing conversations.

Friend-Gated Logic

The system enforces a "friend-gated" model where users can only see full chat histories and receive real-time updates from confirmed friends.

1. **Active Chats:** Conversations with users who are confirmed friends

<code>Frontend/STUDER/src/app/features/messages/components/messages/messages.component.ts</code>	150-151
--	---------

2. **Requests:** Messages from users who are not yet confirmed friends. These are isolated in the "Requests" tab

<code>Frontend/STUDER/src/app/features/messages/components/messages/messages.component.html</code>	45-66
--	-------

3. **Start Chat:** Users can search their `friends` list to initiate a new conversation

<code>Frontend/STUDER/src/app/features/messages/components/messages/messages.component.html</code>	26-43
--	-------

UI States and Placeholders

The component manages several UI states to guide the user:

- **Empty State:** Shown when no conversation is selected (`!selectedChatUserId`)

<code>Frontend/STUDER/src/app/features/messages/components/messages/messages.component.html</code>	107-108
--	---------

- **Request Notice:** Displayed when viewing a message from a non-friend, informing the user they are in `request` mode

<code>Frontend/STUDER/src/app/features/messages/components/messages/messages.component.html</code>	129-131
--	---------

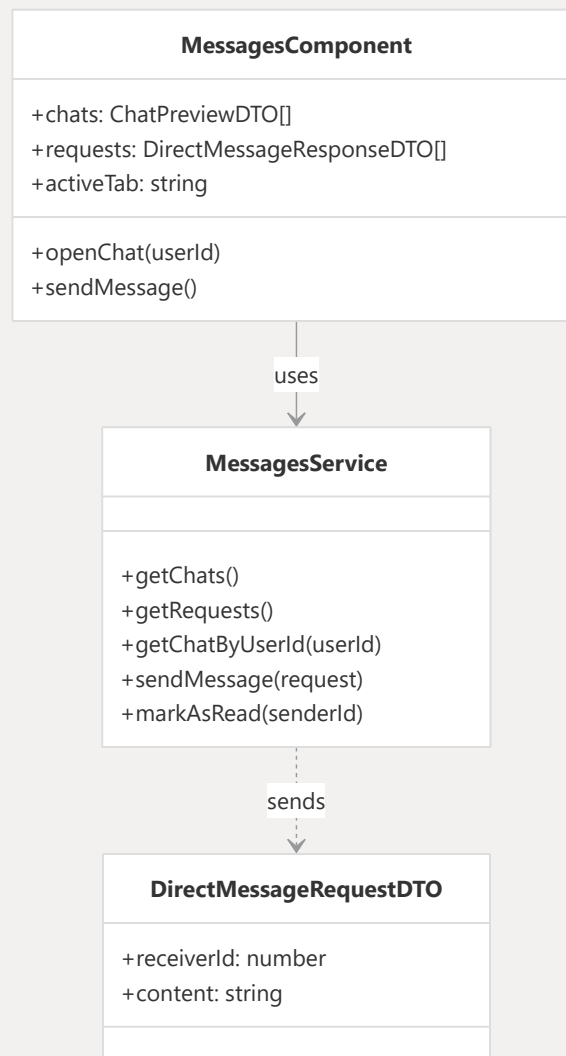
- **Send Placeholder:** The composer input uses a translatable placeholder

`messages.send_placeholder`

Frontend/STUDER/src/app/features/messages/components/messages/messages.component.html

146

Code Entity Mapping



Sources:

Frontend/STUDER/src/app/features/messages/components/messages/messages.component.ts 26-55

Frontend/STUDER/src/app/features/messages/messages.service.ts 13-37

Frontend/STUDER/src/app/features/messages/messages.model.ts 42-47

Implementation Details

Message Sending

The `sendMessage()` function validates the `selectedChatUserId` and content before calling the service. Upon success, it appends the new message to the local `messages` array and refreshes the chat list to update the sidebar preview.

Frontend/STUDER/src/app/features/messages/components/messages/messages.component.ts 157-169

Routing Integration

The component listens for query parameters (`userId`, `username`) to automatically open a specific chat when navigating from a user profile.

Frontend/STUDER/src/app/features/messages/components/messages/messages.component.ts 66-80

Styling

The layout uses a CSS Grid to create a responsive split-pane view:

- `.messages` : Grid container with `320px` sidebar and `1fr` content

Frontend/STUDER/src/app/features/messages/components/messages/messages.component.css 1-8

- `.message--me` : Aligns the current user's messages to the right using `justify-content: flex-end`

Frontend/STUDER/src/app/features/messages/components/messages/messages.component.css 191-193

- `.chat-card__badge` : Displays the `unreadCount` using the primary theme color

Frontend/STUDER/src/app/features/messages/components/messages/messages.component.css 130-136

Sources:

Frontend/STUDER/src/app/features/messages/components/messages/messages.component.css 1-238

Frontend/STUDER/src/app/features/messages/components/messages/messages.component.html 1-152